INT531

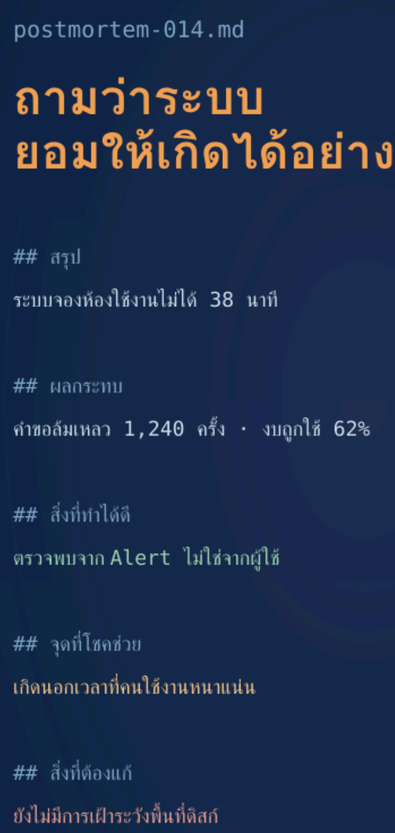
วิศวกรรมความน่าเชื่อถือของระบบ

Blameless Postmortem, Human Factors และวัฒนธรรม SRE

เขียน Postmortem จากเหตุการณ์จริงของกลุ่มตัวเอง แล้วแลกกันตรวจ

คณะเทคโนโลยีสารสนเทศ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

บรรยาย 1.5 ชม. + ปฏิบัติการ 1.5 ชม. | ปรับปรุงเนื้อหา พ.ศ. 2569 (2026)



แผนการเรียนวันนี้

เช้า ทำไมต้องไม่โทษใคร และเขียนอย่างไร · บ่าย เขียนจริงแล้วแลกกันตรวจ

01 **Postmortem มีไว้ทำไม**

เรียนรู้ระดับองค์กร ไม่ใช่การหาคนผิด

02 **โครงสร้างของเอกสาร**

เจ็ดส่วน และหน้าที่ของแต่ละส่วน

03 **ทำไมการโทษคนจึงไม่ได้ผล**

Hindsight Bias, Counterfactual และ Second Story

04 **หาปัจจัยร่วมแทนสาเหตุเดียว**

ถามต่อจนพ้นคำว่าคนทำผิด

05 **Action Item ที่ทำให้อะไรเปลี่ยนจริง**

มีเจ้าของ มีวันที่ และตรวจได้

06 **วัฒนธรรมและจริยธรรม**

ความปลอดภัยทางจิตใจ ความโปร่งใส และความยั่งยืนของทีม

ผลลัพธ์การเรียนรู้ประจำคาบ

เมื่อจบคาบนี้ นักศึกษาจะสามารถ...

SLO 1	เขียน Postmortem ที่มีองค์ประกอบครบตามเทมเพลตมาตรฐาน	**CLO6**
SLO 2	วิเคราะห์สาเหตุโดยไม่หยุดที่ความผิดพลาดของคน	**CLO6**
SLO 3	อธิบาย Blameless Culture, Second Story และ Hindsight Bias	**CLO6, CLO7**
SLO 4	กำหนด Action Item ที่มีเจ้าของ ตรวจได้ และจัดลำดับความสำคัญ	**CLO6**
SLO 5	อภิปรายประเด็นจริยธรรมของผู้ดูแลระบบที่มีผู้ใช้จำนวนมาก	**CLO7**
SLO 6	อธิบายผลกระทบของ Burnout และมาตรการดูแลทีม On-call	**CLO7**

01

Postmortem มีไว้ทำไม

ถ้าเป้าหมายคือหาคนผิด ครั้งหน้าจะไม่มีใครบอกความจริง

วัตถุประสงค์ และสิ่งที่ไม่ใช่วัตถุประสงค์

ความเข้าใจผิดเรื่องนี้ทำให้เอกสารไร้ประโยชน์ตั้งแต่ยังไม่เริ่มเขียน

คือสิ่งนี้

- ทำให้องค์กรเรียนรู้จากเหตุการณ์หนึ่งครั้ง
- หาเงื่อนไขในระบบที่ยอมให้เรื่องนี้เกิดขึ้นได้
- ทำให้คนที่ไม่ได้อยู่ในเหตุการณ์เข้าใจว่าเกิดอะไรขึ้น
- สร้างรายการสิ่งที่ต้องแก้ ที่มีคนรับผิดชอบจริง

ไม่ใช่สิ่งนี้

- การหาว่าใครเป็นคนทำให้เกิด
- เอกสารที่เขียนเพื่อส่งให้ผู้บริหารดูแล้วจบ
- การบันทึกว่าได้ตักเตือนใครไปแล้วบ้าง
- พิธีกรรมที่ทำเพราะกระบวนการกำหนดให้ทำ

เกณฑ์ง่าย ๆ ที่ใช้วัด: ถ้าคนที่อยู่ในเหตุการณ์อ่านแล้วรู้สึกปลอดภัยพอจะเล่าความจริงในครั้งหน้า แปลว่าเขียนถูกทาง

เหตุการณ์แบบใดควรเขียน Postmortem

ตกลงเกณฑ์ไว้ล่วงหน้า เพื่อไม่ต้องถกเถียงกันทีหลัง

☐ ผู้ใช้ได้รับผลกระทบเกินกว่าเกณฑ์ที่ตกลงกันไว้	**เขียนเสมอ**
☐ งบประมาณความผิดพลาดถูกใช้ไปในสัดส่วนที่มีนัยสำคัญ	**เขียนเสมอ**
☐ ต้องใช้การบรรเทาด้วยมือเพื่อให้บริการกลับมา	**เขียนเสมอ**
☐ ตรวจพบช้ากว่าที่ควรจะเป็นมาก	**เขียนเสมอ แม้ผลกระทบจะน้อย**
☐ เกือบเกิดเหตุแต่ไม่เกิด เพราะโชคช่วย	**ควรเขียน และมักมีประโยชน์ที่สุด**

ข้อสุดท้ายคือข้อที่ทีมส่วนใหญ่ข้าม · เหตุการณ์ที่เกือบเกิดคือการเตือนที่ได้มาฟรี โดยยังไม่ต้องจ่ายค่าเสียหาย

02

โครงสร้างของเอกสาร

เจ็ดส่วน แต่ละส่วนมีหน้าที่ของตัวเอง

Postmortem ประกอบด้วยอะไร

และแต่ละส่วนมีไว้ตอบคำถามของใคร

Postmortem ประกอบด้วยอะไร และแต่ละส่วนมีไว้ทำไม

สรุป	สามประโยคที่คนนอกอ่านแล้วเข้าใจ	คนจำนวนมากอ่านแค่ส่วนนี้ส่วนเดียว
ผลกระทบ	ผู้ใช้กี่คน คำขอหายไปเท่าไร งบถูกใช้เท่าไร	เป็นตัวเลข เพื่อเทียบกับเหตุการณ์อื่นได้
ลำดับเหตุการณ์	เกิดอะไรขึ้น และตอนนั้นรู้อะไรบ้าง	แยกสิ่งที่เป็นจริง ออกจากสิ่งที่รู้ ณ ขณะนั้น
ปัจจัยร่วม	เงื่อนไขที่ทำให้เรื่องนี้เกิดขึ้นได้	ใช้คำว่าปัจจัยร่วม เพราะแทบไม่เคยมีสาเหตุเดียว
การตรวจพบและการตอบสนอง	แต่ละขั้นใช้เวลาเท่าไร	บอกว่าจุดอ่อนอยู่ที่การตรวจพบ หรือที่การแก้
ทำได้ดี · ทำได้ไม่ดี · โชคช่วย	สามรายการที่ต้องซื่อสัตย์	ข้อที่สามคือข้อที่คนข้าม และมีประโยชน์ที่สุด
Action Item	มีเจ้าของ มีวันที่ และมีวิธีตรวจว่าทำแล้ว	เป็นส่วนเดียวที่ทำให้อะไรเปลี่ยนจริง

ถ้าเอกสารระบุว่าสาเหตุคือคนคนหนึ่ง แปลว่ายังเขียนไม่เสร็จ · ให้ถามต่อจนกว่าจะได้เงื่อนไขที่ระบบยอมให้เกิด

Timeline — แยกสิ่งที่เป็นจริง ออกจากสิ่งที่รู้

สองคอลัมน์นี้ไม่เหมือนกัน และความต่างคือหัวใจของการวิเคราะห์

สิ่งที่เกิดขึ้นจริง	ดิสก์เต็มตั้งแต่ 02:10 · บริการเริ่มปฏิเสธคำขอตั้งแต่ 02:14
สิ่งที่ทีมรู้ ณ ขณะนั้น	เวลา 02:40 ทีมเพิ่งรู้ว่ามีปัญหา และยังคิดว่าเป็นที่เครือข่าย
ช่องว่างระหว่างสองอย่าง	30 นาทีที่ระบบมีปัญหาโดยไม่มีใครรู้ · นี่คือปัญหาการตรวจจับ ไม่ใช่ปัญหาการแก้
สิ่งที่ใช้วัดผลได้	เวลาตรวจพบ เวลาบรรเทา และเวลากู้คืน แยกกันคนละตัว

การเขียน Timeline ด้วยความรู้ของวันนี้ ทำให้ทุกการตัดสินใจในอดีตดูโง่เขลา · ต้องเขียนว่าตอนนั้นเขารู้อะไร ไม่ใช่สิ่งที่เรารู้ตอนนี้

03

ทำไมการโทษคนจึงไม่ได้ผล

ไม่ใช่เรื่องความใจดี แต่เป็นเรื่องว่าอะไรให้ข้อมูลที่ใช้ได้

สามอคติที่ทำให้วิเคราะห์ผิดทาง

ทั้งสามเกิดขึ้นเองโดยไม่รู้ตัว ต้องตั้งใจป้องกัน

Hindsight Bias	เมื่อรู้ผลแล้ว สัญญาณที่เคยคลุมเครือจะดูชัดเจนย้อนหลัง · ทำให้คิดว่าคนในเหตุการณ์ควรเห็นตั้งแต่แรก
Counterfactual Reasoning	การเขียนว่า ถ้าเขาทำแบบนั้นแทน ก็คงไม่เกิด · ประโยคแบบนี้อธิบายโลกที่ไม่ได้เกิดขึ้น และไม่บอกว่าต้องแก้อะไร
Second Story	เรื่องเล่าที่หนึ่งคือคนทำผิด · เรื่องเล่าที่สองคือทำไมการกระทำนั้นจึงดูสมเหตุสมผลสำหรับเขาในตอนนั้น
หลักที่ใช้ตั้งต้น	สมมติว่าทุกคนตัดสินใจอย่างสมเหตุสมผลจากข้อมูลที่มีอยู่ในขณะนั้น แล้วถามว่าทำไมจึงดูสมเหตุสมผล

แนวคิด Second Story มาจากงานของ Sidney Dekker เรื่องความเข้าใจผิดเกี่ยวกับความผิดพลาดของมนุษย์

เหตุการณ์เดียวกัน ถามสองแบบ

ความต่างอยู่ที่ว่าหยุดถามตรงไหน

หยุดที่ตัวคน

เครื่องหลุดออกจากแผน VLAN

ทำไม? วิศวกรแก้ค่าบนสวิตช์ด้วยมือ

ทำไม? เพราะรีบ

→ สาเหตุ: ความผิดพลาดของคน

→ สิ่งที่จะทำ: เตือนทีมให้ระมัดระวัง

ถามต่อไปจนถึงระบบ

เครื่องหลุดออกจากแผน VLAN

ทำไม? ค่าบนสวิตช์ถูกแก้ด้วยมือ

ทำไม? ไม่มีเส้นทางที่ผ่านการทบทวนให้แก้ค่านั้น

ทำไม? ค่าพอร์ตไม่เคยถูกนำเข้าไปอยู่ในที่เก็บโค้ด

ทำไม? ไม่มีใครเป็นเจ้าของการตั้งค่าสวิตช์ในรูปแบบโค้ด

→ ปัจจัยร่วม: ไม่มีเส้นทางที่ทบทวนได้ ไม่มีเจ้าของ ไม่มีการตรวจเบี่ยงเบน

→ สิ่งที่จะทำ: นำค่าพอร์ตเข้า Git พร้อมการทบทวน มอบหมายเจ้าของ

และแจ้งเตือนเมื่อค่าที่รันอยู่ต่างจากที่เก็บโค้ด

"ให้ระมัดระวังมากขึ้น" ไม่ใช่ Action Item เพราะมอบหมายไม่ได้ กำหนดวันไม่ได้
และตรวจไม่ได้ · คนถัดไปที่เหนื่อยก็จะทำแบบเดิมอีก

04

ปัจจัยร่วมแทนสาเหตุเดียว

ในระบบที่ซับซ้อน แทบไม่เคยมีสาเหตุเดียว

ทำไมจึงไม่ใช้คำว่าสาเหตุราก

คำนี้ชวนให้หยุดค้นหาเร็วเกินไป

เหตุการณ์จริงมักต้องการหลายเงื่อนไขพร้อมกัน	ดิสก์เต็ม ไม่มีการเฝ้าระวังพื้นที่ และ Alert ที่มีก็ถูกปิดเสียงไว้ · ขาดข้อใดข้อหนึ่งก็ไม่เกิด
การหยุดที่สาเหตุเดียวทำให้แก้ไม่ครบ	แก้ดิสก์เต็มอย่างเดียว ครั้งหน้าก็ยังตรวจพบช้าเหมือนเดิม
ถามว่าอะไรทำให้เรื่องนี้เป็นไปได้	แทนที่จะถามว่าใครทำให้เกิด · คำถามแรกได้รายการสิ่งที่แก้ได้ คำถามหลังได้ชื่อคน
หาสิ่งที่ทำให้ตรวจพบช้าด้วย	ไม่ใช่แค่ว่าอะไรทำให้พัง · การตรวจพบช้าคือปัจจัยร่วมที่แยกต่างหาก

เขียนเป็นรายการปัจจัยร่วม แล้วจัดลำดับว่าข้อใดถ้าแก้แล้วตัดโอกาสเกิดซ้ำได้มากที่สุด

05

Action Item

ส่วนเดียวของเอกสารที่ทำให้อะไรเปลี่ยนจริง

Action Item ที่ใช้ได้ กับที่ใช้ไม่ได้

ทดสอบง่าย ๆ คือ มอบหมายได้ไหม กำหนดวันได้ไหม และตรวจได้ไหม

แบบที่ใช้ไม่ได้	ปัญหาคืออะไร	เขียนใหม่ให้ใช้ได้
ให้ระมัดระวังมากขึ้น	มอบหมายไม่ได้ ตรวจไม่ได้ และไม่เปลี่ยนเงื่อนไขใด	เพิ่มการยืนยันก่อนคำสั่งที่ลบข้อมูล โดยแสดงชื่อเครื่องให้เห็น
ปรับปรุงการเฝ้าระวัง	กว้างเกินจนไม่มีใครรู้ว่าเสร็จเมื่อไร	เพิ่ม Alert เมื่อพื้นที่ดิสก์เหลือน้อยกว่า 20% เป็น Ticket ภายในวันที่ ...
อบรมทีมเพิ่ม	โยนภาระไปที่คน ทั้งที่ปัญหาอยู่ที่ระบบ	เขียน Runbook สำหรับขั้นตอนนี้ และใส่ลิงก์ไว้ในตัว Alert
ทบทวนกระบวนการ	ไม่มีเจ้าของและไม่มีผลลัพธ์ที่วัดได้	แก้เกณฑ์การประกาศเหตุการณ์ให้ครอบคลุมกรณีนี้ โดยมีผู้รับผิดชอบคือ ...

จัดลำดับด้วยสองคำถาม: ข้อนี้ลดโอกาสเกิดซ้ำได้มากแค่ไหน และใช้แรงเท่าไร · ข้อที่ได้ผลมากและใช้แรงน้อย ทำก่อนเสมอ

06

วัฒนธรรมและจริยธรรม

กระบวนการที่ดีอยู่ได้ด้วยวัฒนธรรมที่ทำให้คนกล้าพูดความจริง

สิ่งที่ทำให้กระบวนการนี้อยู่ได้จริง

หรือสิ่งที่ทำให้มันกลายเป็นพิธีกรรมภายในสามเดือน

ความปลอดภัยทางจิตใจ	คนต้องเชื่อว่าการเล่าความจริงจะไม่ทำให้ตัวเองเดือดร้อน · ถ้าไม่เชื่อ ข้อมูลที่ได้จะไม่ตรงกับความจริง
ผู้นำต้องเล่าความผิดพลาดของตัวเองก่อน	ถ้าคนที่อาวุโสที่สุดไม่เคยยอมรับว่าพลาด คนอื่นก็จะไม่ยอมรับเช่นกัน
งบประมาณความผิดพลาดเป็นเครื่องมือทางวัฒนธรรม	เปลี่ยนการถกเถียงจากเรื่องความรู้สึก เป็นเรื่องตัวเลขที่ตกลงกันไว้ล่วงหน้า
ความยั่งยืนของทีม	เวรที่ปลุกคนบ่อยเกินไปทำให้คนลาออก · จำนวนครั้งที่ถูกปลุกต่อเวร คือตัวเลขที่ต้องเฝ้าพอ ๆ กับ SLO

สัญญาณเตือนที่ชัดที่สุด: ถ้า Postmortem ฉบับหลัง ๆ สั้นลงเรื่อย ๆ และไม่มีใครเถียงกันในที่ประชุม แปลว่ามันกลายเป็นพิธีกรรมไปแล้ว

ประเด็นจริยธรรมที่ต้องตัดสินใจจริง

ไม่มีคำตอบเดียวที่ถูก แต่ต้องตัดสินใจอย่างรู้ตัวและอธิบายได้

ความโปร่งใสต่อผู้ใช้	บอกผู้ใช้มากแค่ไหนเมื่อระบบล่ม · บอกน้อยไปเสียความไว้วางใจ บอกมากไปอาจเปิดช่องให้ถูกโจมตีซ้ำ
การเปิดเผยเมื่อข้อมูลรั่วไหล	มีภาระตามกฎหมายและตามจริยธรรม · ความเร็วในการแจ้งสำคัญต่อผู้ได้รับผลกระทบมากกว่าความครบถ้วน
การไม่ระบุตัวบุคคลต่อสาธารณะ	รายงานสาธารณะไม่ควรระบุว่าใครเป็นคนพิมพ์คำสั่ง · องค์กรเป็นผู้รับผิดชอบ ไม่ใช่พนักงานคนเดียว
ความรับผิดชอบต่อผู้ใช้จำนวนมาก	ระบบที่มีผู้ใช้หลายพันคน การตัดสินใจทางเทคนิคกลายเป็นการตัดสินใจที่กระทบคนจำนวนมาก

ในบริบทของเรา: ระบบของคณะมีผู้ใช้เป็นนักศึกษาและบุคลากรจริง การตัดสินใจเหล่านี้จึงไม่ใช่เรื่องสมมติ

ปฏิบัติการ — เขียนแล้วแลกกันตรวจ

หกขั้น ใช้เหตุการณ์ที่บันทึกไว้สัปดาห์ที่แล้ว



เช็กลิสต์การตรวจเอกสารของอีกกลุ่ม

ใช้ตรวจแล้วส่งคืนพร้อมความเห็น

☐	สรุปสามประโยคอ่านแล้วเข้าใจ โดยไม่ต้องรู้บริบทมาก่อน	**มี / ไม่มี**
☐	ผลกระทบเป็นตัวเลข ไม่ใช่คำบรรยาย	**มี / ไม่มี**
☐	Timeline แยกสิ่งที่เกิดขึ้นจริง ออกจากสิ่งที่ทีมรู้ตอนนั้น	**มี / ไม่มี**
☐	ไม่มีประโยคที่ระบุว่าใครเป็นสาเหตุ	**ตรวจทุกบรรทัด**
☐	มีปัจจัยร่วมมากกว่าหนึ่งข้อ	**นับจำนวน**
☐	ทุก Action Item มีเจ้าของ วันที่ และวิธีตรวจว่าทำแล้ว	**นับข้อที่ขาด**

ถ้าเจอประโยคที่โทษบุคคล ให้เขียนข้อเสนอว่าควรเขียนใหม่อย่างไร ไม่ใช่แค่ชี้ว่าผิด · นั่นคือการตรวจแบบ Blameless เช่นกัน

สรุปสามประโยคของวันนี้

01

ถ้าเป้าหมายคือหาคนผิด ครั้งหน้าจะไม่มีใครบอกความจริง · เอกสารที่ทำให้คนกลัว จะได้ข้อมูลที่บิดเบี้ยวและเรียนรู้อะไรไม่ได้

02

ในระบบที่ซับซ้อนแทบไม่เคยมีสาเหตุเดียว · ถามว่าอะไรทำให้เรื่องนี้เป็นไปได้ จะได้รายการสิ่งที่แก้ได้ ส่วนการถามว่าใครทำ จะได้แค่ชื่อคน

03

Action Item คือส่วนเดียวที่ทำให้อะไรเปลี่ยนจริง · ข้อที่มอบหมายไม่ได้ กำหนดวันไม่ได้ และตรวจไม่ได้ เท่ากับไม่ได้เขียนไว้เลย

ส่งรายงาน Blameless Postmortem ฉบับสมบูรณ์ คิดเป็น 10% ของรายวิชา

เอกสารอ้างอิง

เรียงตามลำดับที่ควรอ่าน

ตำราและเอกสารหลัก

- Beyer, B. et al. (2016). Site Reliability Engineering. O'Reilly. บทที่ 15 (Postmortem Culture)
- Dekker, S. The Field Guide to Understanding 'Human Error' — บทคัดสรรเรื่อง Second Story
- Allspaw, J. บทความเรื่อง Blameless Postmortems และ Just Culture
- เทมเพลต Postmortem ของรายวิชา

สำหรับอ่านต่อ

- คลังรายงาน Postmortem สาธารณะของบริษัทเทคโนโลยี
- รายงานเหตุการณ์ของ Reddit, Datadog, GitLab และ Cloudflare ที่ใช้เป็นกรณีศึกษาในสัปดาห์ก่อน ๆ
- Google SRE Workbook บทที่ว่าด้วย Postmortem Culture